

DOSSIER UML

Application Point de Vente (POS)

Débites de Boissons

Stack technique : Django (REST API) · PostgreSQL · Flutter (Dart)

Équipe GSI — 2026

Table des matières

1. Introduction et contexte du projet
2. Diagrammes de cas d'utilisation
 - 2.1 Vue d'ensemble
 - 2.2 Module Administration
 - 2.3 Module Point de Vente (POS)
 - 2.4 Module Gestion Commerciale
 - 2.5 Module Achats & Stocks
 - 2.6 Module Comptabilité & Rapports
3. Diagramme de classes
4. Diagramme Entité-Relation (ERD)
5. Diagrammes de séquence
 - 5.1 Authentification (JWT)
 - 5.2 Création d'une vente (POS)
6. Diagramme d'activité — Processus d'achat
7. Synthèse architecturale

1. Introduction et contexte du projet

Ce dossier UML décrit l'architecture fonctionnelle et technique de l'application Point de Vente (POS) destinée aux établissements de vente de boissons (débits de boissons). Il regroupe l'ensemble des diagrammes UML produits dans le cadre du projet académique, classés du plus structurant au plus détaillé.

L'application est conçue pour couvrir les besoins opérationnels complets d'un bar ou d'un débit de boissons : enregistrement des ventes au comptoir, gestion des stocks et des achats fournisseurs, suivi comptable des dépenses, ainsi que l'administration des utilisateurs et des paramètres système.

Stack technique

Composant	Technologie
Backend / API REST	Django ≥ 5.0 + Django REST Framework
Base de données	PostgreSQL (schéma géré via schema.sql, managed=False)
Authentification	JWT (JSON Web Token) via djangorestframework-simplejwt
Frontend mobile/web	Flutter (Dart)
Diagrammes UML	PlantUML · StarUML

Modules fonctionnels

Le système est structuré en six modules fonctionnels principaux :

- Administration : gestion des utilisateurs, rôles, permissions et paramètres système
- Point de Vente (POS) : saisie des ventes, gestion des clients et des paiements
- Gestion Commerciale : catalogue produits, catégories, fournisseurs
- Achats & Stocks : commandes fournisseurs, réception, mouvements de stock
- Comptabilité & Rapports : suivi des dépenses, tableau de bord, rapports
- Authentification transversale : connexion sécurisée par JWT pour tous les utilisateurs

2. Diagrammes de cas d'utilisation

Les diagrammes de cas d'utilisation décrivent les interactions entre les acteurs (utilisateurs du système) et les fonctionnalités offertes par l'application. Six vues complémentaires ont été produites : une vue d'ensemble et cinq vues par module. Les acteurs principaux sont : Administrateur, Gérant, Comptable, Caissier, Serveur et Magasinier — tous héritant de l'acteur générique Utilisateur via une relation de généralisation.

2.1 Vue d'ensemble

Ce diagramme synthétise toutes les interactions entre les acteurs et les grands blocs fonctionnels de l'application. Il illustre la hiérarchie des acteurs via la généralisation vers l'acteur Utilisateur, qui centralise l'accès au cas d'utilisation S'authentifier. Chaque acteur dispose ensuite d'accès spécifiques aux modules selon son rôle métier.

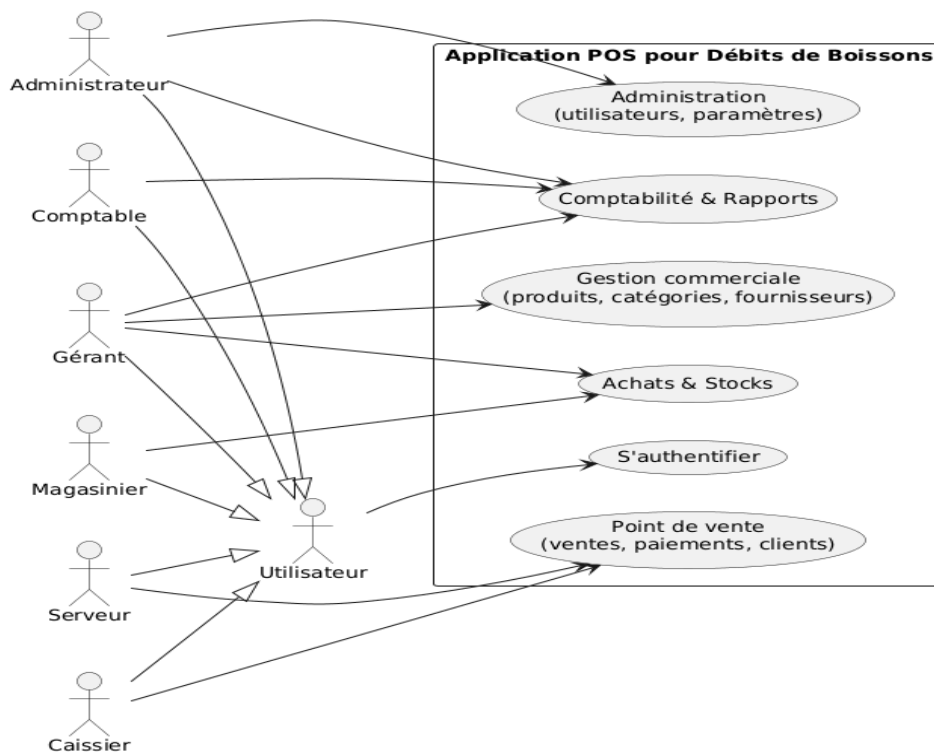


Figure 1 — Vue d'ensemble des cas d'utilisation (tous acteurs)

Acteur	Modules accessibles
Administrateur	Administration, Comptabilité & Rapports
Gérant	Gestion Commerciale, Achats & Stocks, Comptabilité & Rapports
Comptable	Comptabilité & Rapports
Caissier / Serveur	Point de Vente (ventes, paiements, clients)
Magasinier	Achats & Stocks

2.2 Module Administration

Le module Administration est réservé exclusivement à l'Administrateur. Il lui permet de gérer le cycle de vie des utilisateurs (création, modification, désactivation), d'attribuer des rôles — cas inclus dans la gestion des utilisateurs — et de configurer les paramètres globaux de l'application. L'Administrateur peut également consulter le tableau de bord et générer des rapports d'activité.

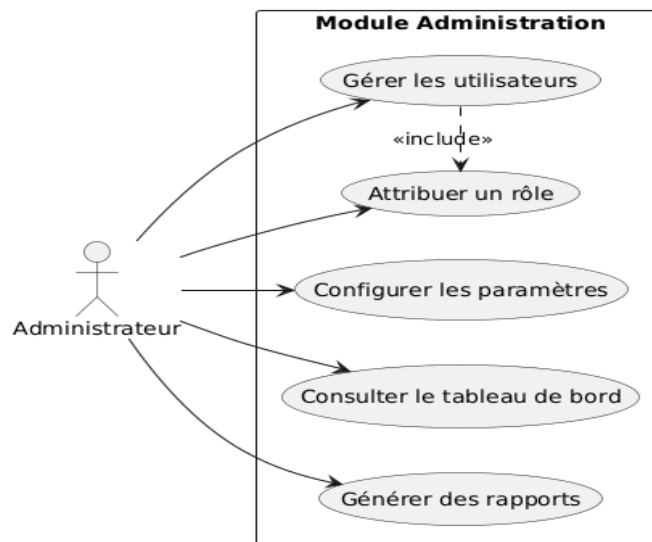


Figure 2 — Module Administration

- Gérer les utilisateurs «include» Attribuer un rôle
- Configurer les paramètres (horaires, TVA, modes de paiement acceptés, etc.)
- Consulter le tableau de bord
- Générer des rapports

2.3 Module Point de Vente (POS)

Ce module est au cœur de l'activité quotidienne de l'établissement. Il est utilisé par le Caissier et le Serveur. La gestion des ventes (POS) inclut systématiquement la gestion des paiements via une relation «include», garantissant qu'aucune vente n'est enregistrée sans validation du paiement associé. La gestion des clients permet de fidéliser la clientèle et d'associer des ventes à des clients identifiés.

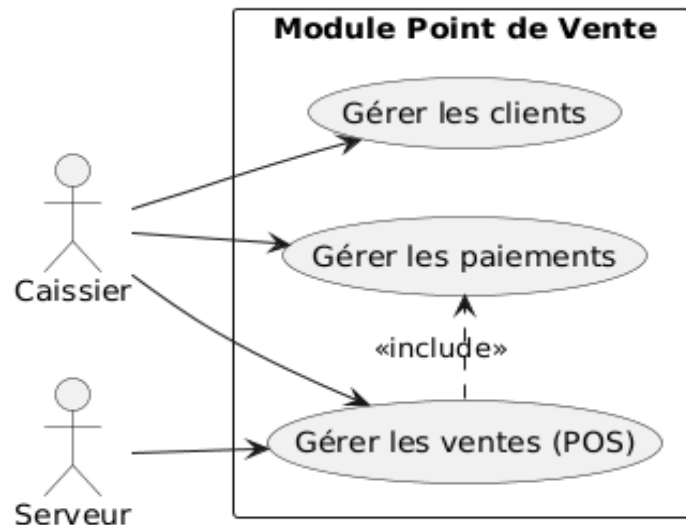


Figure 3 — Module Point de Vente

- Gérer les clients : création et consultation des fiches clients
- Gérer les paiements : espèces, carte bancaire, mobile money
- Gérer les ventes (POS) «include» Gérer les paiements

2.4 Module Gestion Commerciale

Ce module est piloté par le Gérant. Il couvre la gestion du catalogue produits et des catégories (boissons alcoolisées, soft drinks, snacks...), ainsi que la gestion des fournisseurs. La relation «include» entre Gérer les produits et Gérer les catégories traduit le fait qu'un produit est obligatoirement rattaché à une catégorie. Le Gérant accède également au tableau de bord et aux rapports.

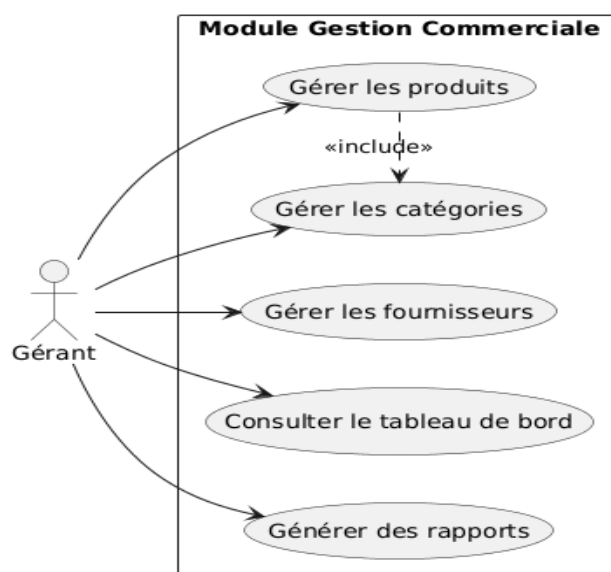


Figure 4 — Module Gestion Commerciale

2.5 Module Achats & Stocks

Ce module implique le Gérant et le Magasinier. La gestion des stocks est accessible aux deux acteurs, tandis que la gestion des achats (passation de commandes fournisseurs) est une fonctionnalité incluse dans le processus plus large de gestion des stocks — chaque achat génère un mouvement de stock entrant. Cette conception reflète le flux physique réel : commande → réception → mise à jour du stock.

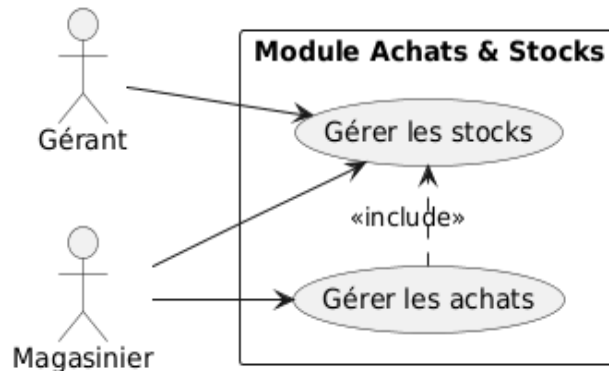


Figure 5 — Module Achats & Stocks

- Gérer les stocks : consultation des niveaux, alertes de seuil, mouvements
- Gérer les achats «include» dans Gérer les stocks

2.6 Module Comptabilité & Rapports

Ce module est partagé entre le Comptable, le Gérant et l'Administrateur. Le Comptable gère les dépenses opérationnelles (loyer, salaires, charges diverses). La génération de rapports est accessible au Gérant et à l'Administrateur, avec une relation «extend» depuis la consultation du tableau de bord, ce qui signifie que les rapports constituent une extension approfondie des indicateurs affichés.

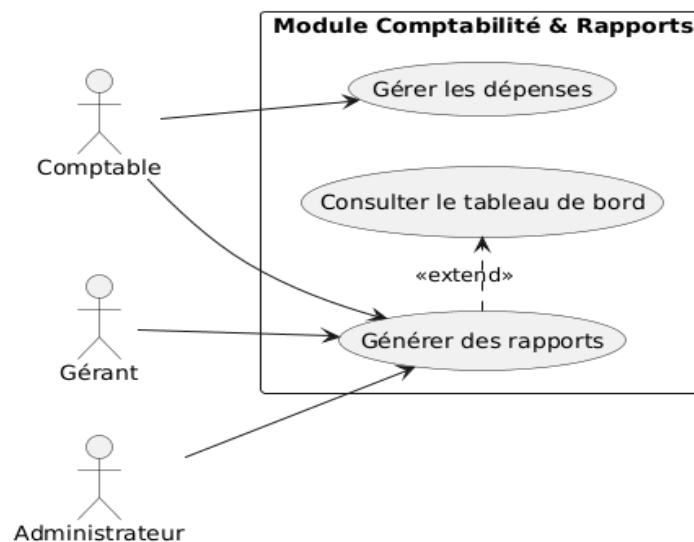


Figure 6 — Module Comptabilité & Rapports

- Gérer les dépenses (Comptable uniquement)
- Consulter le tableau de bord
- Générer des rapports «extend» Consulter le tableau de bord

3. Diagramme de classes

Le diagramme de classes constitue la colonne vertébrale de l'architecture logicielle. Il décrit les entités métier, leurs attributs et les relations structurales (associations, agrégations, dépendances) entre elles. Ce modèle est directement transposé en modèles Django (fichiers models.py) avec l'option `managed = False`, le schéma physique étant géré indépendamment via PostgreSQL.

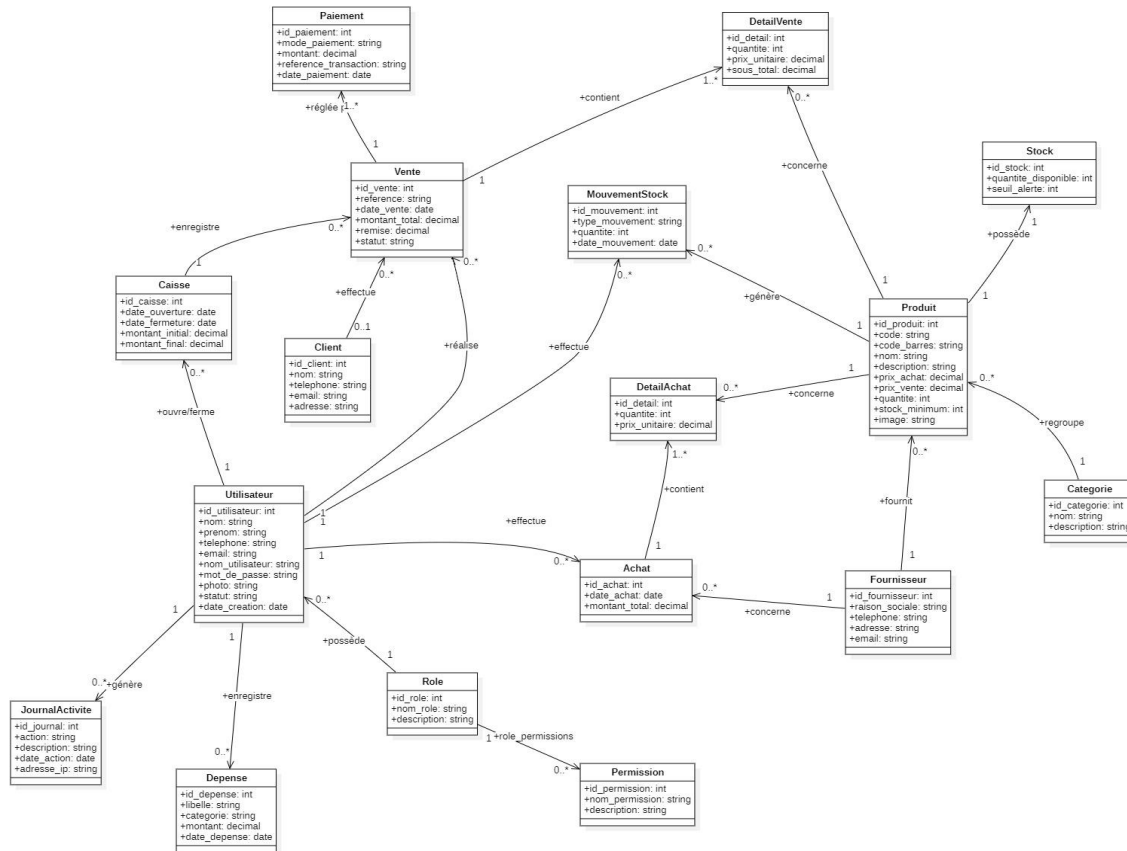


Figure 7 — Diagramme de classes complet (modèle objet)

Entités principales

Entité	Rôle et attributs clés
Utilisateur	Acteur central du système. Attributs : id, nom, prénom, email, nom_utilisateur, mot_de_passe (haché), photo, statut, date_création. Lié à un Role.
Role / Permission	Modèle RBAC (Role-Based Access Control). Un utilisateur possède un rôle ; un rôle agrège des permissions. Permet une gestion fine des droits.
Produit	Élément du catalogue. Attributs : code, code_barres, nom, description, prix_achat, prix_vente, quantite, stock_minimum, image. Appartient à une Categorie.
Categorie	Regroupe les produits (ex. : Bières, Vins, Soft drinks, Snacks).
Fournisseur	Partenaire commercial. Attributs : raison_sociale, téléphone, adresse, email.
Vente	Transaction de caisse. Attributs : référence, date_vente, montant_total, remise, statut. Liée à un Client optionnel et à un Utilisateur (caissier).
DetailVente	Ligne de vente. Attributs : quantité, prix_unitaire, sous_total (calculé). Concerne un Produit.
Paiement	Règlement d'une vente. Attributs : mode_paiement, montant, référence_transaction, date_paiement.
Achat	Commande fournisseur. Attributs : date_achat, montant_total. Effectuée par un Utilisateur, concerne un Fournisseur.
DetailAchat	Ligne d'achat. Attributs : quantité, prix_unitaire. Concerne un Produit.

Stock	Niveau de stock par produit. Attributs : quantite_disponible, seuil_alerte.
MouvementStock	Traçabilité des flux. Attributs : type_mouvement (entrée/sortie/ajustement), quantité, date_mouvement.
Caisse	Session de caisse. Attributs : date_ouverture, date_fermeture, montant_initial, montant_final. Ouverte/fermée par un Utilisateur.
Depense	Charge opérationnelle. Attributs : libellé, catégorie, montant, date.
Client	Client de l'établissement. Attributs : nom, téléphone, email, adresse.
JournalActivite	Audit trail. Attributs : action, description, date_action, adresse_ip. Généré par un Utilisateur.

Relations clés

- Utilisateur (1) ↔ Role (1) : chaque utilisateur possède exactement un rôle
- Vente (1) → DetailVente (1..*) : une vente contient au moins une ligne
- Achat (1) → DetailAchat (1..*) : un achat contient au moins une ligne
- Produit (1) ↔ Stock (1) : relation un-à-un pour le suivi des niveaux
- MouvementStock généré à chaque vente (sortie) et chaque réception d'achat (entrée)
- Caisse (0..1) enregistre (0..*) Ventes et (0..*) Dépenses

4. Diagramme Entité-Relation (ERD)

Le diagramme Entité-Relation représente le schéma physique de la base de données PostgreSQL. Il complète le diagramme de classes en précisant les clés primaires (PK), les types de données SQL, et les cardinalités de chaque relation. Ce schéma est la source de vérité du projet : il est déployé directement via un fichier `schema.sql`, et les modèles Django utilisent `managed = False` pour ne pas interférer avec sa structure.

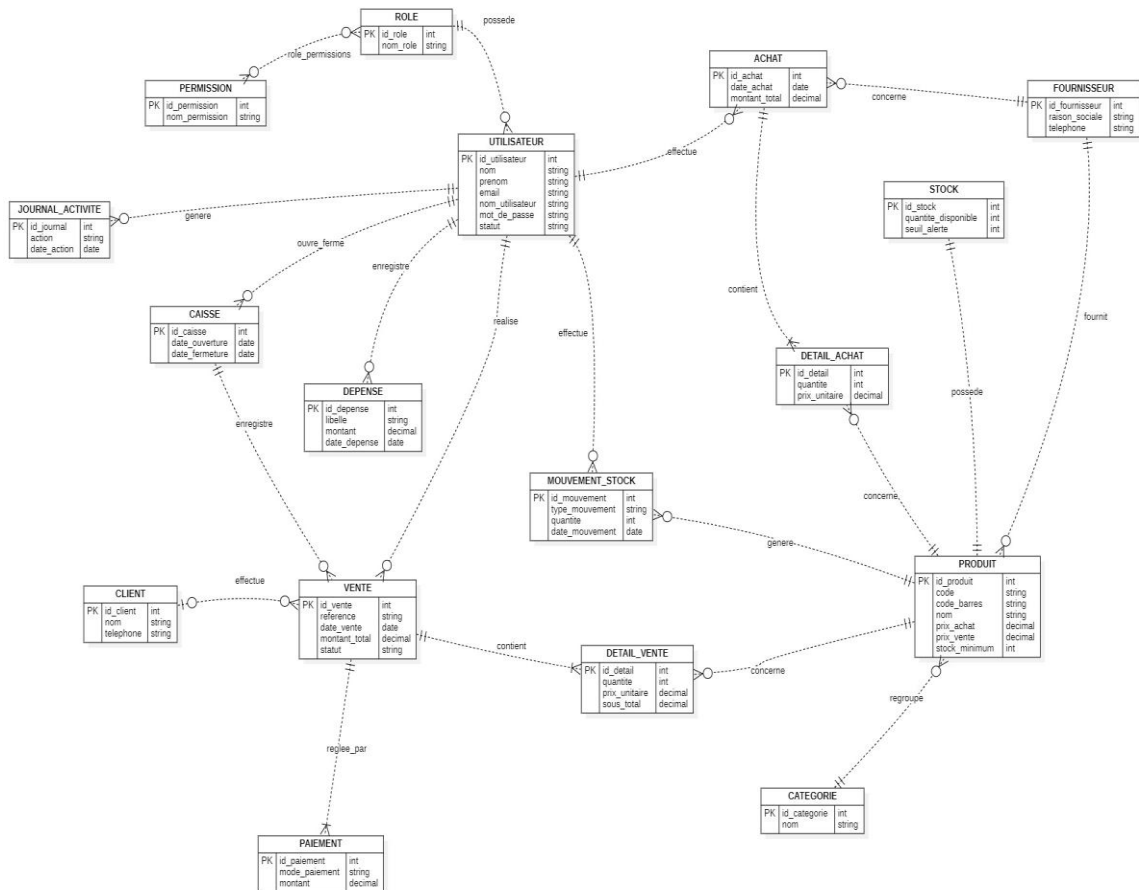


Figure 8 — Diagramme Entité-Relation (schéma PostgreSQL)

Observations clés sur le schéma physique

- Toutes les clés primaires sont de type INT (séquences PostgreSQL ou SERIAL).
- Les montants monétaires sont de type DECIMAL pour éviter les erreurs d'arrondi en virgule flottante.
- La colonne `sous_total` de `DETAIL_VENTE` est une colonne générée (GeneratedField Django / GENERATED ALWAYS AS PostgreSQL), calculée automatiquement comme `quantite × prix_unitaire`.
- Les relations entre tables sont modélisées par des clés étrangères avec contraintes d'intégrité référentielle.
- La table `MOUVEMENT_STOCK` est mise à jour de façon atomique à chaque opération de vente ou de réception d'achat (`transaction.atomic()` + `select_for_update()` côté Django).
- Les tables `ROLE` et `PERMISSION` implémentent un RBAC qui peut être intégré au système de permissions Django ou géré manuellement selon les choix de l'équipe.

5. Diagrammes de séquence

Les diagrammes de séquence décrivent les interactions dynamiques entre les composants du système dans le temps. Ils illustrent l'ordre des messages échangés entre l'acteur, le frontend Flutter, l'API REST Django et la base de données PostgreSQL pour deux flux critiques : l'authentification et la création d'une vente.

5.1 Authentification par JWT

Le flux d'authentification repose sur le standard JWT (JSON Web Token). Lorsqu'un utilisateur saisit ses identifiants dans l'application Flutter, ceux-ci sont transmis via un POST /auth/login à l'API Django. Le backend vérifie l'existence de l'utilisateur en base, contrôle le mot de passe haché (bcrypt ou PBKDF2), puis génère un token JWT signé en cas de succès. Ce token est stocké localement par le frontend et joint à chaque requête ultérieure dans l'en-tête Authorization: Bearer <token>. En cas d'identifiants invalides, l'API retourne une réponse HTTP 401 Unauthorized.

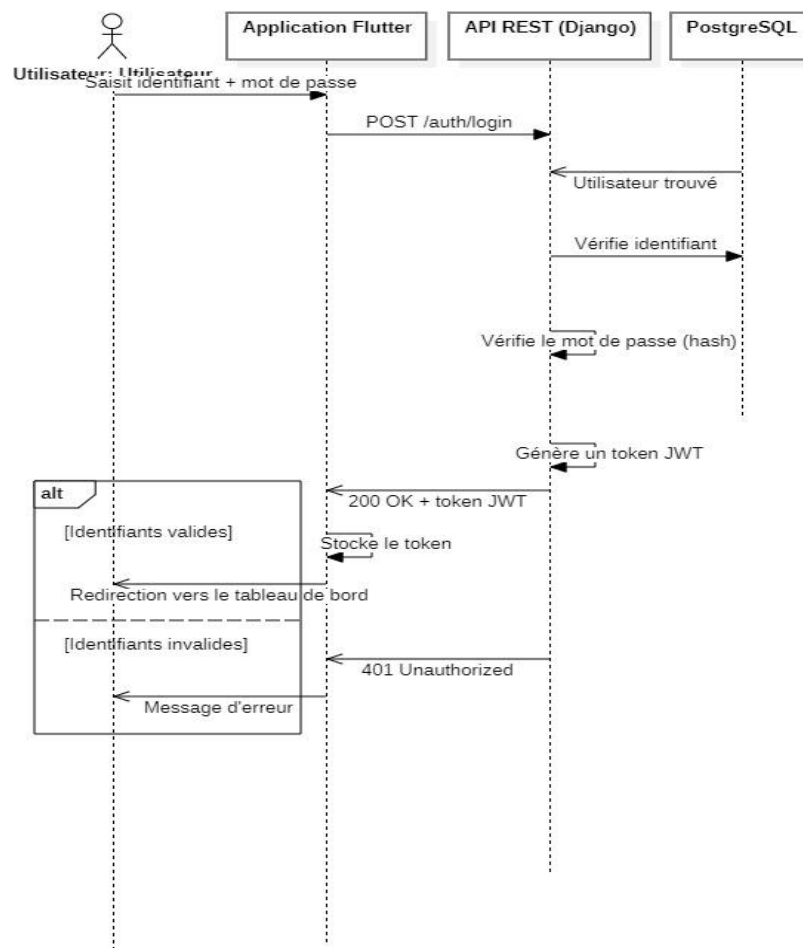


Figure 9 — Diagramme de séquence : Authentification JWT

Étape	Description
1. Saisie	L'utilisateur entre identifiant + mot de passe dans l'interface Flutter
2. POST /auth/login	Flutter envoie les credentials à l'API REST Django
3. Lookup utilisateur	Django interroge PostgreSQL pour trouver l'utilisateur
4. Vérification hash	Django compare le mot de passe avec le hash stocké
5a. Succès	Génération d'un token JWT → 200 OK → stockage local → redirection tableau de bord
5b. Échec	401 Unauthorized → affichage du message d'erreur dans Flutter

5.2 Création d'une vente (POS)

Ce flux représente le cas d'utilisation central de l'application : l'enregistrement d'une transaction de caisse. Le caissier scanne ou sélectionne un produit, ce qui déclenche un GET /produits/{id} pour récupérer les informations produit (prix, disponibilité). Le panier est géré localement dans Flutter. À la validation du paiement, un POST /ventes transmet la vente complète à Django, qui exécute de façon atomique : l'insertion de la vente et de ses lignes, la décrémentation du stock produit, et l'insertion d'un mouvement de stock. Le ticket de caisse (reçu) est ensuite retourné au frontend pour impression.

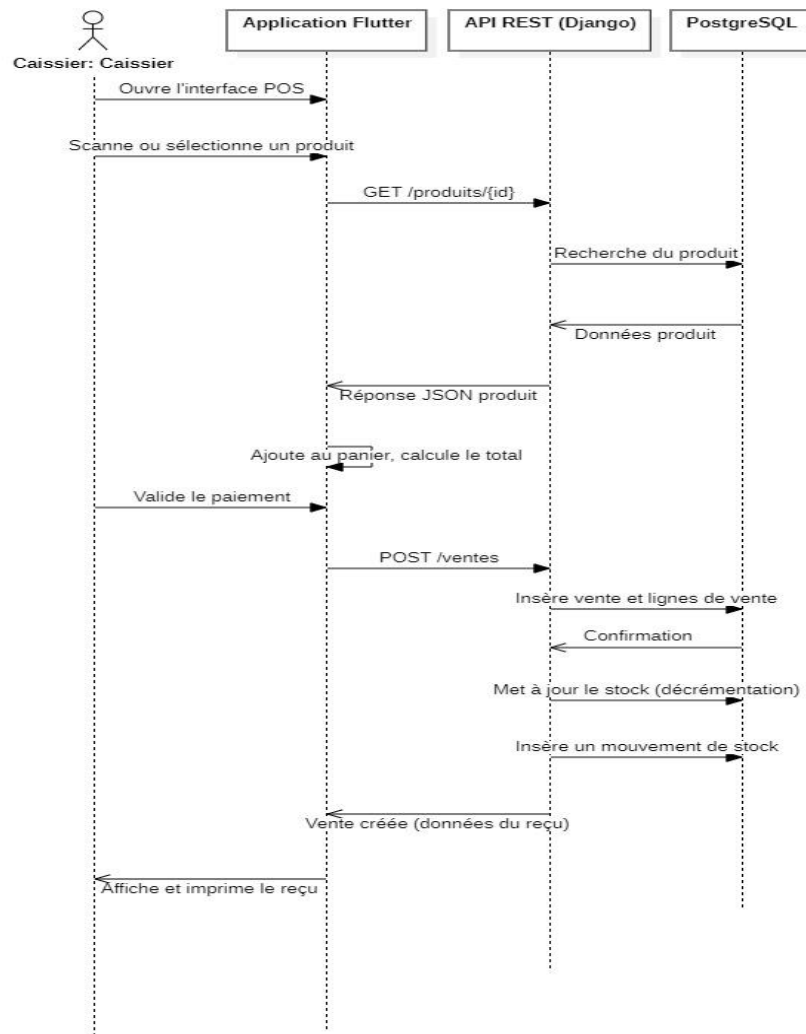


Figure 10 — Diagramme de séquence : Création d'une vente (POS)

- Atomicité : les opérations INSERT vente + UPDATE stock + INSERT mouvement sont dans une seule transaction PostgreSQL
- Concurrency : select_for_update() sur les lignes de stock pour éviter les races conditions en cas d'utilisation multi-caisses
- Le reçu retourné contient la référence de vente, les lignes, les totaux et le mode de paiement

6. Diagramme d'activité — Processus d'achat fournisseur

Le diagramme d'activité (flowchart) modélise le processus métier complet de passation et réception d'une commande fournisseur. Ce flux est déclenché par le Gérant ou le Magasinier lorsque le stock d'un ou plusieurs produits atteint le seuil d'alerte défini.

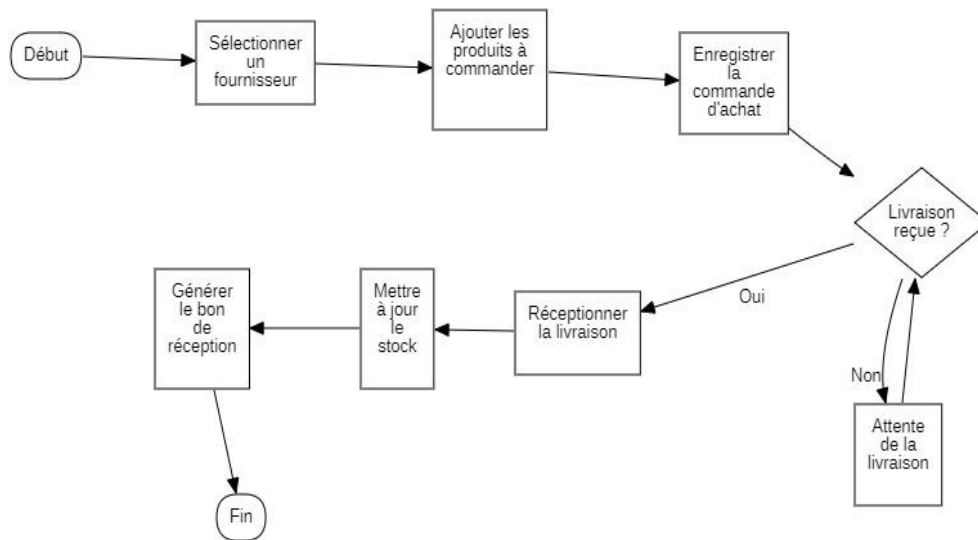


Figure 11 — Diagramme d'activité : Processus d'achat fournisseur

Description des étapes

Étape	Description
Sélectionner un fournisseur	Choix du fournisseur dans le référentiel (module Gestion Commerciale)
Ajouter les produits à commander	Sélection des produits et saisie des quantités commandées
Enregistrer la commande d'achat	Création de l'entité Achat et ses DetailAchat en base de données
Livraison reçue ? (décision)	Boucle d'attente jusqu'à réception physique de la marchandise
Réceptionner la livraison	Confirmation de la réception, contrôle des quantités reçues
Mettre à jour le stock	Incrémentation du stock et insertion d'un MouvementStock de type 'entrée'
Générer le bon de réception	Édition du document de réception (traçabilité comptable et logistique)

Ce processus est critique pour l'intégrité des stocks : toute réception doit générer un mouvement de stock entrant, afin de maintenir la cohérence entre le stock physique et le stock informatique.

7. Synthèse architecturale

L'ensemble des diagrammes produits forme un dossier UML cohérent qui couvre les quatre dimensions de la modélisation UML : structurelle (classes, ERD), comportementale (cas d'utilisation, activité), dynamique (séquence) et déploiement (implicite via la séparation Flutter / Django / PostgreSQL).

Cohérence inter-diagrammes

- Les entités du diagramme de classes correspondent exactement aux tables du ERD et aux `models.py` Django
- Les flux des diagrammes de séquence s'appuient sur les endpoints REST identifiés dans les vues Django (`views.py`)
- Les acteurs des cas d'utilisation correspondent aux rôles définis dans les tables `ROLE` et `PERMISSION`
- Le diagramme d'activité détaille le cas d'utilisation 'Gérer les achats' identifié dans la vue Module Achats & Stocks

Décisions architecturales majeures

Décision	Justification
managed = False sur tous les modèles Django	Le schéma PostgreSQL est la source de vérité ; les migrations Django ne gèrent pas le DDL
GeneratedField pour sous_total	Calcul automatique <code>quantite × prix_unitaire</code> garanti par PostgreSQL, sans logique applicative dupliquée
transaction.atomic() + select_for_update()	Garantie d'atomicité et de cohérence en contexte multi-utilisateurs (plusieurs caisses simultanées)
JWT pour l'authentification	Standard moderne, stateless, adapté aux API REST consommées par un client mobile Flutter
Séparation Caisse / Vente	La Caisse représente une session de travail ; une Vente est une transaction individuelle dans cette session

Équipe projet

Membre	Rôle / Modules
Gildas	Lead analyste UML + co-développeur (ventes, achats, stocks, paiements, dépenses)
Paulo	Co-développeur — modules opérationnels + Frontend Flutter
Verone	Architecte technique — scaffolding du projet
Michel	Base de données + authentification
Yohan	Authentification / Sécurité JWT
Patricia	Gestion commerciale (produits, catégories, fournisseurs, clients)
Laetitia	Rapports, tableau de bord, tests, déploiement
Bouba & OSSOM	Frontend Flutter (équipe GSA)